

Data and Insights

The original data set used for this project had a wide range of characteristics. These characteristics ranged from Animal ID to color. However, it required significant preprocessing before it could be used for machine learning. The data, stored as string or object types in a CSV file, contained missing values and redundancies that needed to be addressed.

The process of cleaning the data began by removing all of the 17 duplicate rows to ensure the model doesn't become biased. Unusable variables that would not help machine learning, such as Animal ID and Name, were also dropped. These columns don't provide valuable information regarding the animals and offer no predictive patterns for the machine learning models. Lastly, the Breed column was removed due to the wide range of values for breed, which would lead to complications in the machine learning process.

The final part of data preparation involved converting the object data types into usable numeric data. To create valuable numeric features, I created several functions and applied them to the data. Before functions can be applied, however, I had to edit the categorical columns containing missing data. These missing values were replaced with the mode of that column grouped by Animal Type.

Using functions, I created the following categories to usable numeric data. Using the date of birth column, the exact age in years (float) for each animal was stored. Next, the MonthYear and DateTime objects were used to extract the exact month, date, and year animals were admitted to the shelter (float). Finally, the OutAge column was used to calculate the age of the animals when they left the shelter in days.

After converting the numeric data types, the rest of the non-numeric data columns were converted to categorical data and then into numeric data through one-hot encoding. The Color category, however, had to be engineered further before it could be converted into categorical data. There were too many different color combinations. I took the first part of each color combination to be the animal's primary color, reducing the number of color entries. This step significantly reduced the number of distinct color categories, making the data more manageable and improving the model's ability to generalize. What is noticeable, however, is that the dependent variable, "Outcome Type," was manually altered in place rather than through the `get_dummies()` function. Here, I wanted to maintain the dependent variable as a single category where "Adoption" was assigned a value of 1 while "Transfer" was assigned a value of 0.

The data preparation phase revealed many interesting insights. Univariate analysis, in particular, provided crucial insights into the data's distribution. Although the data initially appeared clean, it actually contained numerous inconsistencies, including missing entries, formatting issues, and variables that provided little analytical value. Univariate analysis also revealed that several features contained outliers, especially within the Age category. Additionally, the dataset was imbalanced, with more animals being adopted than transferred, which could influence model performance. In contrast, the MonthAdmit and DayAdmit features

were evenly distributed, indicating animals were admitted consistently throughout the year. Ultimately, the data preparation process highlighted the importance of crucial data cleaning and feature engineering for successful machine learning. Raw data often has many inconsistencies and unusable data/formatting. Moreover, the dataset can naturally contain biases and outliers. Consequently, data preparation and data exploration is a stage that is essential to creating a good model that can capture the pattern you want the model to analyze.

Procedure

To train the model, the dependent variable and independent variables were extracted. The dependent variable was “Outcome Type,” while the independent variable was everything but the “Outcome Type”. Next, the variables were split into two groups, one for training the model and one for testing/validation. Seventy percent of the data was reserved for training data, while the last thirty percent was used for validation. What is significant is that in the data split, an equal ratio of dependent variables in both categories was kept through “stratify” to ensure that the model fits correctly. The model was then trained using supervised classification algorithms such as K-Nearest Neighbors using 5 nearest neighbors, Linear Classification, and KNN using GridSearchCV with 5-fold cross-validation (Hyperparameter range of 1 to 99 nearest neighbors).

Confidence and Performance

Each of the models was evaluated using accuracy, recall, precision, and F1-scores to assess their performance. The K-Nearest Neighbors classifier and the GridSearchCV both outperformed the linear classifier. Both of the KNN models achieved a training accuracy of about 0.86 and a test accuracy of 0.80, indicating strong performances. In contrast, the linear classifier achieved an accuracy of approximately 0.68 on both training and test data, showing that it was less capable of capturing the patterns in the dataset.

The most important metric for this data set is recall. This is because the shelter wants to capture adoption trends and have a better understanding of adoption success for future reference and planning. Penalizing false negatives is important to ensure that the shelter has all the data necessary to plan future adoptions. If the model misses true adoptions, the shelter might underestimate adoption success and misjudge factors that help animals get adopted. The recall score for both KNN was 0.94 on the training set and about 0.90 on the testing set, meaning that the model was able to correctly identify the majority of true adoptions. This is valuable as understanding adoption trends and correctly recognizing successful adoptions is key to assessing program effectiveness and planning for future resource allocation and promotion. Consequently, I have high confidence in the model. On the other hand, the Linear Classification model did very poorly with a recall score of around .6 for both the test and the training model. Consequently, I don't have much confidence in that model.

Overall, the results suggest that the KNN-based models provide reliable and consistent performance, making them suitable for predicting adoption outcomes with confidence.